

Name: MILLION SOLOMON

ID: 501280237

EE8222 W25

Assignment 1: Hadoop,Hive, and SparkSQL

1. Downloading the dataset and creating a table called used_cars from the data

```
%sh
wget https://bit.ly/ClassifiedCars -O cars.zip
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 2 sec. Last updated by anonymous at February 04 2025, 2:34:36 PM. (outdated)

```
%sh
unzip cars.zip -d cars_data
```

ERROR ▶ ⌵ ⌵ ⌵ ⌵

Took 0 sec. Last updated by anonymous at February 04 2025, 2:47:07 PM.

```
%sh
hadoop fs -mkdir -p /user/hive/warehouse/cars_dataset
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 5 sec. Last updated by anonymous at February 04 2025, 2:38:32 PM. (outdated)

```
%sh
hadoop fs -put /var/lib/zeppelin/cars_data/cars.csv /user/hive/warehouse/cars_dataset/
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 5 sec. Last updated by anonymous at February 04 2025, 3:00:10 PM. (outdated)

```
%hive
CREATE DATABASE IF NOT EXISTS car_analysis;
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 0 sec. Last updated by anonymous at February 04 2025, 4:38:01 PM.

```
%hive
USE car_analysis;
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 0 sec. Last updated by anonymous at February 04 2025, 4:38:01 PM.

```
%hive
CREATE EXTERNAL TABLE IF NOT EXISTS car_analysis.used_cars (
  maker STRING,
  model STRING,
  mileage INT,
  manufacture_year INT,
  engine_displacement INT,
  engine_power INT,
  body_type STRING,
  color_slug STRING,
  stk_year INT,
  transmission STRING,
  door_count INT,
  seat_count INT,
  fuel_type STRING,
  date_created TIMESTAMP,
  date_last_seen TIMESTAMP,
  price_eur DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/hive/warehouse/cars_dataset'
TBLPROPERTIES ("skip.header.line.count"="1");
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

Took 1 sec. Last updated by anonymous at February 04 2025, 4:40:21 PM.

```
%hive
SELECT
  SUM(CASE WHEN maker IS NULL OR maker = '' THEN 1 ELSE 0 END) AS maker_nulls,
  SUM(CASE WHEN model IS NULL OR model = '' THEN 1 ELSE 0 END) AS model_nulls,
  SUM(CASE WHEN mileage IS NULL THEN 1 ELSE 0 END) AS mileage_nulls,
  SUM(CASE WHEN manufacture_year IS NULL THEN 1 ELSE 0 END) AS manufacture_year_nulls,
  SUM(CASE WHEN engine_displacement IS NULL THEN 1 ELSE 0 END) AS engine_displacement_nulls,
  SUM(CASE WHEN engine_power IS NULL THEN 1 ELSE 0 END) AS engine_power_nulls,
  SUM(CASE WHEN body_type IS NULL OR body_type = '' THEN 1 ELSE 0 END) AS body_type_nulls,
  SUM(CASE WHEN color_slug IS NULL OR color_slug = '' THEN 1 ELSE 0 END) AS color_slug_nulls,
  SUM(CASE WHEN stk_year IS NULL THEN 1 ELSE 0 END) AS stk_year_nulls,
  SUM(CASE WHEN transmission IS NULL OR transmission = '' THEN 1 ELSE 0 END) AS transmission_nulls,
  SUM(CASE WHEN door_count IS NULL THEN 1 ELSE 0 END) AS door_count_nulls,
  SUM(CASE WHEN seat_count IS NULL THEN 1 ELSE 0 END) AS seat_count_nulls,
  SUM(CASE WHEN fuel_type IS NULL OR fuel_type = '' THEN 1 ELSE 0 END) AS fuel_type_nulls,
  SUM(CASE WHEN date_created IS NULL THEN 1 ELSE 0 END) AS date_created_nulls,
  SUM(CASE WHEN date_last_seen IS NULL THEN 1 ELSE 0 END) AS date_last_seen_nulls,
  SUM(CASE WHEN price_eur IS NULL THEN 1 ELSE 0 END) AS price_nulls
FROM car_analysis.used_cars_fixed;
```

FINISHED ▶ ⌵ ⌵ ⌵ ⌵

2. Queries to find how many missing values (NULL in an attribute of a record) you have in each column (attribute)

```
thrive
SELECT
  SUM(CASE WHEN maker IS NULL OR maker = '' THEN 1 ELSE 0 END) AS maker_nulls,
  SUM(CASE WHEN model IS NULL OR model = '' THEN 1 ELSE 0 END) AS model_nulls,
  SUM(CASE WHEN mileage IS NULL THEN 1 ELSE 0 END) AS mileage_nulls,
  SUM(CASE WHEN manufacture_year IS NULL THEN 1 ELSE 0 END) AS manufacture_year_nulls,
  SUM(CASE WHEN engine_displacement IS NULL THEN 1 ELSE 0 END) AS engine_displacement_nulls,
  SUM(CASE WHEN engine_power IS NULL THEN 1 ELSE 0 END) AS engine_power_nulls,
  SUM(CASE WHEN body_type IS NULL OR body_type = '' THEN 1 ELSE 0 END) AS body_type_nulls,
  SUM(CASE WHEN color_slug IS NULL OR color_slug = '' THEN 1 ELSE 0 END) AS color_slug_nulls,
  SUM(CASE WHEN stk_year IS NULL THEN 1 ELSE 0 END) AS stk_year_nulls,
  SUM(CASE WHEN transmission IS NULL OR transmission = '' THEN 1 ELSE 0 END) AS transmission_nulls,
  SUM(CASE WHEN door_count IS NULL THEN 1 ELSE 0 END) AS door_count_nulls,
  SUM(CASE WHEN seat_count IS NULL THEN 1 ELSE 0 END) AS seat_count_nulls,
  SUM(CASE WHEN fuel_type IS NULL OR fuel_type = '' THEN 1 ELSE 0 END) AS fuel_type_nulls,
  SUM(CASE WHEN date_created IS NULL THEN 1 ELSE 0 END) AS date_created_nulls,
  SUM(CASE WHEN date_last_seen IS NULL THEN 1 ELSE 0 END) AS date_last_seen_nulls,
  SUM(CASE WHEN price_eur IS NULL THEN 1 ELSE 0 END) AS price_nulls
FROM car_analysis.used_cars_fixed;
```

FINISHED ▶ ⌵ ⌵ ⌵

Exported as Excel file

maker_nulls	518915
model_nulls	1133361
mileage_nulls	362584
manufacture_year_nulls	370578
engine_displacement_nulls	743414
engine_power_nulls	554877
body_type_nulls	1122914
color_slug_nulls	3343411
stk_year_nulls	3016807
transmission_nulls	741630
door_count_nulls	1090066
seat_count_nulls	1287099
fuel_type_nulls	1847606
date_created_nulls	0
date_last_seen_nulls	0
price_nulls	0

```
thrive
WITH null_counts AS (
  SELECT
    COUNT(*) AS total_records,
    ROUND(100 * SUM(CASE WHEN maker IS NULL OR maker = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS maker_null_pct,
    ROUND(100 * SUM(CASE WHEN model IS NULL OR model = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS model_null_pct,
    ROUND(100 * SUM(CASE WHEN mileage IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS mileage_null_pct,
    ROUND(100 * SUM(CASE WHEN manufacture_year IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS manufacture_year_null_pct,
    ROUND(100 * SUM(CASE WHEN engine_displacement IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS engine_displacement_null_pct,
    ROUND(100 * SUM(CASE WHEN engine_power IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS engine_power_null_pct,
    ROUND(100 * SUM(CASE WHEN body_type IS NULL OR body_type = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS body_type_null_pct,
    ROUND(100 * SUM(CASE WHEN color_slug IS NULL OR color_slug = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS color_slug_null_pct,
    ROUND(100 * SUM(CASE WHEN stk_year IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS stk_year_null_pct,
    ROUND(100 * SUM(CASE WHEN transmission IS NULL OR transmission = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS transmission_null_pct,
    ROUND(100 * SUM(CASE WHEN door_count IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS door_count_null_pct,
    ROUND(100 * SUM(CASE WHEN seat_count IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS seat_count_null_pct,
    ROUND(100 * SUM(CASE WHEN fuel_type IS NULL OR fuel_type = '' THEN 1 ELSE 0 END) / COUNT(*), 2) AS fuel_type_null_pct,
    ROUND(100 * SUM(CASE WHEN date_created IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS date_created_null_pct,
    ROUND(100 * SUM(CASE WHEN date_last_seen IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS date_last_seen_null_pct,
    ROUND(100 * SUM(CASE WHEN price_eur IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS price_null_pct
  FROM car_analysis.used_cars_fixed
)
-- First Table: Display NULL Percentage for Each Column
SELECT
  'maker' AS column_name, maker_null_pct AS null_percentage FROM null_counts UNION ALL
SELECT 'model', model_null_pct FROM null_counts UNION ALL
SELECT 'mileage', mileage_null_pct FROM null_counts UNION ALL
SELECT 'manufacture_year', manufacture_year_null_pct FROM null_counts UNION ALL
SELECT 'engine_displacement', engine_displacement_null_pct FROM null_counts UNION ALL
SELECT 'engine_power', engine_power_null_pct FROM null_counts UNION ALL
SELECT 'body_type', body_type_null_pct FROM null_counts UNION ALL
SELECT 'color_slug', color_slug_null_pct FROM null_counts UNION ALL
SELECT 'stk_year', stk_year_null_pct FROM null_counts UNION ALL
SELECT 'transmission', transmission_null_pct FROM null_counts UNION ALL
SELECT 'door_count', door_count_null_pct FROM null_counts UNION ALL
SELECT 'seat_count', seat_count_null_pct FROM null_counts UNION ALL
SELECT 'fuel_type', fuel_type_null_pct FROM null_counts UNION ALL
SELECT 'date_created', date_created_null_pct FROM null_counts UNION ALL
SELECT 'date_last_seen', date_last_seen_null_pct FROM null_counts UNION ALL
SELECT 'price_eur', price_eur_null_pct FROM null_counts
ORDER BY null_percentage DESC;
-- Second Table: Display columns with 2 98% NULLs to be dropped
SELECT column_name, null_percentage
FROM (
  SELECT 'maker' AS column_name, maker_null_pct AS null_percentage FROM null_counts UNION ALL
  SELECT 'model', model_null_pct FROM null_counts UNION ALL
  SELECT 'mileage', mileage_null_pct FROM null_counts UNION ALL
  SELECT 'manufacture_year', manufacture_year_null_pct FROM null_counts UNION ALL
  SELECT 'engine_displacement', engine_displacement_null_pct FROM null_counts UNION ALL
  SELECT 'engine_power', engine_power_null_pct FROM null_counts UNION ALL
  SELECT 'body_type', body_type_null_pct FROM null_counts UNION ALL
  SELECT 'color_slug', color_slug_null_pct FROM null_counts UNION ALL
  SELECT 'stk_year', stk_year_null_pct FROM null_counts UNION ALL
  SELECT 'transmission', transmission_null_pct FROM null_counts UNION ALL
  SELECT 'door_count', door_count_null_pct FROM null_counts UNION ALL
  SELECT 'seat_count', seat_count_null_pct FROM null_counts UNION ALL
  SELECT 'fuel_type', fuel_type_null_pct FROM null_counts UNION ALL
  SELECT 'date_created', date_created_null_pct FROM null_counts UNION ALL
  SELECT 'date_last_seen', date_last_seen_null_pct FROM null_counts UNION ALL
  SELECT 'price_eur', price_eur_null_pct FROM null_counts
) tmp
WHERE null_percentage >= 98
ORDER BY null_percentage DESC;
```

ERROR ▶ ⌵ ⌵ ⌵

column_name	null_percentage
color_slug	94.1
skt_year	84.91
fuel_type	52.0
seat_count	36.23
model	31.9
body_type	31.61
door_count	30.68
engine_displacement	20.52
transmission	20.87
engine_power	15.62
maker	14.61
manufacture_year	10.43
mileage	10.21
price_eur	0.0
date_last_seen	0.0
date_created	0.0

3. Dropping the columns (attribute) with more than 50% missing values

```

-- Hive
ALTER TABLE car_analysis.used_cars_fixed
REPLACE COLUMNS (
  maker STRING,
  model STRING,
  mileage INT,
  manufacture_year INT,
  engine_displacement INT,
  engine_power INT,
  body_type STRING,
  transmission STRING,
  door_count INT,
  seat_count INT,
  price_eur DOUBLE,
  date_created TIMESTAMP,
  date_last_seen TIMESTAMP
);

```

col_name	data_type	comment
# col_name	data_type	comment
maker	string	
model	string	
mileage	int	
manufacture_year	int	
engine_displacement	int	
engine_power	int	
body_type	string	
transmission	string	
door_count	int	
seat_count	int	
price_eur	double	
date_created	timestamp	
date_last_seen	timestamp	

4. Creating a new table called clean_used_cars from used_cars by filtering.

```

-- Hive
CREATE TABLE car_analysis.clean_used_cars AS
SELECT
  maker,
  model,
  mileage,
  manufacture_year,
  engine_displacement,
  engine_power,
  body_type,
  transmission,
  door_count,
  seat_count,
  price_eur,
  date_created,
  date_last_seen
FROM car_analysis.used_cars_cleaned
WHERE
  manufacture_year BETWEEN 2000 AND 2017
  AND maker IS NOT NULL AND maker != ''
  AND model IS NOT NULL AND model != ''
  AND price_eur BETWEEN 3000 AND 2000000;

```

5. Records in clean_used_cars

```

-- Hive
SELECT COUNT(*) AS total_records
FROM car_analysis.clean_used_cars;

```

total_records
1322853

6. Make and model for the cars with the top 10 highest average prices

Query: Top 10 Make & Models with Highest Average Prices

Finished

Took 0 sec. Last updated by anonymous at February 05 2025, 12:20:53 PM.

```
hive
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM car_analysis.clean_used_cars
GROUP BY maker, model
ORDER BY avg_price DESC
LIMIT 10;
```

Finished

maker	model	avg_price
lamborghini	aventador	365961.0
porsche	carrera-gt	302045.22
bmw	z8	245118.6
tesla	roadster	192880.28
tesla	model-x	176418.32
bentley	brooklands	138501.3
rolls-royce	wraith	137663.47
bentley	continental-gtc	129138.88
bmw	i8	112273.43
bentley	continental-gt	105946.89

7. Make and model for the cars with the top 10 lowest average prices.

Query: Top 10 Make & Models with Lowest Average Prices

Finished

Took 0 sec. Last updated by anonymous at February 05 2025, 12:30:57 PM.

```
hive
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM car_analysis.clean_used_cars
GROUP BY maker, model
ORDER BY avg_price ASC -- Sorting in ascending order for lowest prices
LIMIT 10;
```

Finished

maker	model	avg_price
skoda	galaxy	3071.8
rover	streetwise	3187.85
kia	retona	3200.25
bmw	transit	3290.16
chevrolet	alero	3305.37
hyundai	santamo	3391.01
opel	kadett	3405.48
fiat	128	3460.4
nissan	frontier	3478.9
seat	inca	3498.66

8. Recommending the top five make and models for **Economic Segment** customers

Query: Top 5 Make & Models for Economic Segment

Finished

Took 0 sec. Last updated by anonymous at February 05 2025, 12:36:13 PM.

```
hive
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM car_analysis.clean_used_cars
WHERE price_eur BETWEEN 3000 AND 20000 -- Economic segment price range
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;
```

maker	model	avg_price
volvo	241	19980.0
volvo	960	19306.14
toyota	gt86	18807.61
toyota	venza	18510.29
infinity	m30	18424.13

9. Recommending the top five make and models for **Intermediate Segment** customers

Query: Top 5 Make & Models for Intermediate Segment FINISHED ▶ 🔍 ⚙️

Took 0 sec. Last updated by anonymous at February 05 2025, 12:40:35 PM.

```

thive
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM car_analysis.clean_used_cars
WHERE price_eur BETWEEN 30000 AND 300000 -- Intermediate segment price range
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;

```

FINISHED ▶ 🔍 ⚙️

maker	model	avg_price
lamborghini	aventador	272901.63
bmw	z8	235838.12
tesla	model-x	176418.32
hyundai	matrix	143241.43
bentley	brooklands	138501.3

10. Recommending the top five make and models for the **Luxury Segment** customers

Query: Top 5 Make & Models for Luxury Segment FINISHED ▶ 🔍 ⚙️

Took 0 sec. Last updated by anonymous at February 05 2025, 12:47:14 PM.

```

thive
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM car_analysis.clean_used_cars
WHERE price_eur BETWEEN 300000 AND 2000000 -- Luxury segment price range
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;

```

READY ▶ 🔍 ⚙️

maker	model	avg_price
mazda	323	1350749.44
volkswagen	golf	1179236.37
fiat	500	1125000.0
toyota	yaris	1111152.11
fiat	panda	1100000.0

Assignment-1-SparkSQL

FINISHED

```
## Name: MILLION SOLOMON
## ID: 501280237
```

Name: MILLION SOLOMON

ID: 501280237

Took 0 sec. Last updated by anonymous at February 12 2025, 1:48:23 PM.

```
%md
### Downloading and transferring the dataset to HDFS
```

FINISHED

Downloading and transferring the dataset to HDFS

Took 0 sec. Last updated by anonymous at February 12 2025, 1:49:50 PM.

```
%sh
wget https://bit.ly/ClassifiedCars -O cars.zip
unzip cars.zip && rm cars.zip
```

FINISHED

```
--2025-02-12 18:36:00-- https://bit.ly/ClassifiedCars
Resolving bit.ly (bit.ly)... 67.199.248.10, 67.199.248.11
Connecting to bit.ly (bit.ly)|67.199.248.10|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://github.com/tofighi/dataset/raw/main/big-data/cars.zip [following]
--2025-02-12 18:36:00-- https://github.com/tofighi/dataset/raw/main/big-data/cars.zip
Resolving github.com (github.com)... 140.82.112.3
Connecting to github.com (github.com)|140.82.112.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/tofighi/dataset/main/big-data/cars.zip [following]
--2025-02-12 18:36:01-- https://raw.githubusercontent.com/tofighi/dataset/main/big-data/cars.zip
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 81295459 (78M) [application/zip]
Saving to: 'cars.zip\r'
```

Output is truncated to 102400 bytes. Learn more about ZEPPELIN_INTERPRETER_OUTPUT_LIMIT

Took 9 sec. Last updated by anonymous at February 12 2025, 1:36:06 PM.

```
%sh
ls -l
```

FINISHED

```
total 489040
drwxr-xr-x 2 zeppelin zeppelin 4096 Nov 15 2021 __MACOSX
-rwxr-xr-x 1 zeppelin zeppelin 419466302 Sep 20 2019 cars.csv
-rw-r--r-- 1 zeppelin zeppelin 81295459 Feb 12 18:36 cars.zip
```

drwxr-xr-x 3 zeppelin zeppelin 4096 Feb 4 19:36 cars_data
drwxr-xr-x 7 zeppelin zeppelin 4096 Jan 25 00:31 notebook

Took 0 sec. Last updated by anonymous at February 12 2025, 1:36:21 PM.

Assignment-1-SparkSQL

```
%sh
hdfs dfs -mkdir -p /user/zeppelin/datasets
hdfs dfs -put -f cars.csv /user/zeppelin/datasets/
```

FINISHED

Took 13 sec. Last updated by anonymous at February 12 2025, 1:37:58 PM.

```
%sh
hdfs dfs -ls /user/zeppelin/datasets/
```

FINISHED

Found 1 items

```
-rw-r--r-- 2 zeppelin hadoop 419466302 2025-02-12 18:37 /user/zeppelin/datasets/cars.csv
```

Took 4 sec. Last updated by anonymous at February 12 2025, 1:44:14 PM.

```
%md
### Creating the used_cars Table in the Default Database
```

FINISHED

Creating the used_cars Table in the Default Database

Took 0 sec. Last updated by anonymous at February 12 2025, 1:56:22 PM.

```
%sql
DROP TABLE IF EXISTS used_cars;

CREATE TABLE used_cars USING CSV
OPTIONS (
  path '/user/zeppelin/datasets/cars.csv',
  header 'true',
  inferSchema 'true',
  delimiter ',',
);
```

SPARK JOB FINISHED

Took 1 min 3 sec. Last updated by anonymous at February 12 2025, 1:57:43 PM.

```
%sql
SHOW TABLES;
```

FINISHED

       settings

namespace	tableName	isTemporary
default	clean_used_cars_dummy	false
default	used_cars	false

Assignment-1-SparkSQL

Took 1 sec. Last updated by anonymous at February 12 2025, 1:58:21 PM. (outdated)

%sql
SELECT * FROM used_cars LIMIT 10;

SPARK JOB (http://hadoop-m.us-central1-c.c.ee8222-w25-449918.internal:35857/jobs/job?id=2) FINISHED

settings

maker	model	mileage	manufacture_year	engine_displacement	engine_power	body_type	color :
ford	galaxy	151000	2011	2000	103	null	null
skoda	octavia	143476	2012	2000	81	null	null
bmw	null	97676	2010	1995	85	null	null
skoda	fabia	111970	2004	1200	47	null	null
skoda	fabia	128886	2004	1200	47	null	null
skoda	fabia	140932	2003	1200	40	null	null
skoda	fabia	167220	2001	1400	74	null	null
bmw	null	148500	2009	2000	130	null	null

Took 1 sec. Last updated by anonymous at February 12 2025, 1:58:53 PM. (outdated)

Count NULL Values for Each Column

FINISHED

Took 0 sec. Last updated by anonymous at February 12 2025, 2:01:15 PM.

%sql
SELECT
COUNT(*) AS total_records,

SUM(CASE WHEN maker IS NULL OR maker = '' OR maker = 'None' THEN 1 ELSE 0 END) AS maker_nulls,
SUM(CASE WHEN model IS NULL OR model = '' OR model = 'None' THEN 1 ELSE 0 END) AS model_nulls,
SUM(CASE WHEN mileage IS NULL OR mileage = 'None' THEN 1 ELSE 0 END) AS mileage_nulls,
SUM(CASE WHEN manufacture_year IS NULL OR manufacture_year = 'None' THEN 1 ELSE 0 END) AS manufacture_year_nulls,
SUM(CASE WHEN engine_displacement IS NULL OR engine_displacement = 'None' THEN 1 ELSE 0 END) AS engine_displacement_nulls,
SUM(CASE WHEN engine_power IS NULL OR engine_power = 'None' THEN 1 ELSE 0 END) AS engine_power_nulls,

SUM(CASE WHEN body_type IS NULL OR body_type = '' OR body_type = 'None' THEN 1 ELSE 0 END) AS body_type_nulls,
SUM(CASE WHEN color_slug IS NULL OR color_slug = '' OR color_slug = 'None' THEN 1 ELSE 0 END) AS color_slug_nulls,
SUM(CASE WHEN stk_year IS NULL OR stk_year = 'None' THEN 1 ELSE 0 END) AS stk_year_nulls,
SUM(CASE WHEN transmission IS NULL OR transmission = '' OR transmission = 'None' THEN 1 ELSE 0 END) AS transmission_nulls,

SUM(CASE WHEN door_count IS NULL OR door_count = 'None' THEN 1 ELSE 0 END) AS door_count_nulls,
SUM(CASE WHEN seat_count IS NULL OR seat_count = 'None' THEN 1 ELSE 0 END) AS seat_count_nulls,

SUM(CASE WHEN fuel_type IS NULL OR fuel_type = '' OR fuel_type = 'None' THEN 1 ELSE 0 END) AS fuel_type_nulls,
SUM(CASE WHEN price_eur IS NULL OR price_eur = 'None' THEN 1 ELSE 0 END) AS price_nulls,

SUM(CASE WHEN CAST(NULLIF(date_created, 'None') AS TIMESTAMP) IS NULL THEN 1 ELSE 0 END) AS date_created_nulls,
SUM(CASE WHEN CAST(NULLIF(date_last_seen, 'None') AS TIMESTAMP) IS NULL THEN 1 ELSE 0 END) AS date_last_seen_nulls

FROM used_cars;

SPARK JOB FINISHED

settings ▾

Assignment-1-SparkSQL

model_nulls ▾	mileage_nulls ▾	manufacture_year_nulls ▾	engine_displacement_nulls ▾	engine_pov
3552912	518915	1133361	362584	370578
			743414	554877 ▲

Took 55 sec. Last updated by anonymous at February 12 2025, 2:06:42 PM. (outdated)

```
%sql
WITH null_counts AS (
  SELECT
    COUNT(*) AS total_records,

    ROUND(100 * SUM(CASE WHEN maker IS NULL OR maker = '' OR maker = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS maker_null_pct,
    ROUND(100 * SUM(CASE WHEN model IS NULL OR model = '' OR model = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS model_null_pct,
    ROUND(100 * SUM(CASE WHEN mileage IS NULL OR mileage = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS mileage_null_pct,
    ROUND(100 * SUM(CASE WHEN manufacture_year IS NULL OR manufacture_year = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS manufacture_year_null_pct,
    ROUND(100 * SUM(CASE WHEN engine_displacement IS NULL OR engine_displacement = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS engine_displacement_null_pct,
    ROUND(100 * SUM(CASE WHEN engine_power IS NULL OR engine_power = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS engine_power_null_pct,

    ROUND(100 * SUM(CASE WHEN body_type IS NULL OR body_type = '' OR body_type = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS body_type_null_pct,
    ROUND(100 * SUM(CASE WHEN color_slug IS NULL OR color_slug = '' OR color_slug = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS color_slug_null_pct,
    ROUND(100 * SUM(CASE WHEN stk_year IS NULL OR stk_year = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS stk_year_null_pct,
    ROUND(100 * SUM(CASE WHEN transmission IS NULL OR transmission = '' OR transmission = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS transmission_null_pct,

    ROUND(100 * SUM(CASE WHEN door_count IS NULL OR door_count = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS door_count_null_pct,
    ROUND(100 * SUM(CASE WHEN seat_count IS NULL OR seat_count = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS seat_count_null_pct,

    ROUND(100 * SUM(CASE WHEN fuel_type IS NULL OR fuel_type = '' OR fuel_type = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS fuel_type_null_pct,
    ROUND(100 * SUM(CASE WHEN price_eur IS NULL OR price_eur = 'None' THEN 1 ELSE 0 END) / COUNT(*), 2) AS price_null_pct,

    ROUND(100 * SUM(CASE WHEN CAST(NULLIF(date_created, 'None')) AS TIMESTAMP) IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS date_created_null_pct,
    ROUND(100 * SUM(CASE WHEN CAST(NULLIF(date_last_seen, 'None')) AS TIMESTAMP) IS NULL THEN 1 ELSE 0 END) / COUNT(*), 2) AS date_last_seen_null_pct

  FROM used_cars
)

SELECT column_name, null_percentage
FROM (
  SELECT 'maker' AS column_name, maker_null_pct AS null_percentage FROM null_counts UNION ALL
  SELECT 'model', model_null_pct FROM null_counts UNION ALL
  SELECT 'mileage', mileage_null_pct FROM null_counts UNION ALL
  SELECT 'manufacture_year', manufacture_year_null_pct FROM null_counts UNION ALL
  SELECT 'engine_displacement', engine_displacement_null_pct FROM null_counts UNION ALL
  SELECT 'engine_power', engine_power_null_pct FROM null_counts UNION ALL
  SELECT 'body_type', body_type_null_pct FROM null_counts UNION ALL
  SELECT 'color_slug', color_slug_null_pct FROM null_counts UNION ALL
  SELECT 'stk_year', stk_year_null_pct FROM null_counts UNION ALL
  SELECT 'transmission', transmission_null_pct FROM null_counts UNION ALL
  SELECT 'door_count', door_count_null_pct FROM null_counts UNION ALL
  SELECT 'seat_count', seat_count_null_pct FROM null_counts UNION ALL
  SELECT 'fuel_type', fuel_type_null_pct FROM null_counts UNION ALL
  SELECT 'date_created', date_created_null_pct FROM null_counts UNION ALL
  SELECT 'date_last_seen', date_last_seen_null_pct FROM null_counts UNION ALL
  SELECT 'price_eur', price_null_pct FROM null_counts
) AS null_data
```

SPARK JOB FINISHED

```
ORDER BY null_percentage DESC;
```

Assignment-1-SparkSQL

column_name	nuui_percentage
color_slug	94.1
stk_year	84.91
fuel_type	52.0
seat_count	36.23
model	31.9
body_type	31.61
door_count	30.68
engine_displacement	20.92

Took 1 min 33 sec. Last updated by anonymous at February 12 2025, 2:13:13 PM. (outdated)

Columns to Drop From your NULL percentage results, the following columns have $\geq 50\%$ missing values:

1.color_slug

2.stk_year

3.fuel_type

Took 0 sec. Last updated by anonymous at February 12 2025, 2:15:53 PM.

```
%md
### Dropping Columns with More Than 50% Missing Values.
```

Dropping Columns with More Than 50% Missing Values.

Took 0 sec. Last updated by anonymous at February 12 2025, 2:20:12 PM.

```
%sql
CREATE OR REPLACE TEMPORARY VIEW cars_cleaned AS
SELECT
  maker,
  model,
  mileage,
  manufacture_year,
  engine_displacement,
  engine_power,
  body_type,
  transmission,
  door_count,
  seat_count,
  price_eur,

  -- Convert 'None' values to NULL and cast as TIMESTAMP
  CAST(NULLIF(date_created, 'None') AS TIMESTAMP) AS date_created,
  CAST(NULLIF(date_last_seen, 'None') AS TIMESTAMP) AS date_last_seen
```

FROM used_cars;

Took 0 sec. Last updated by anonymous at February 12 2025, 2:31:28 PM.

Assignment-1-SparkSQL

SPARK JOB (http://hadoop-m.us-central1-c.c.ee8222-w25-449918.internal:35857/jobs/job?id=24) FINISHED

```
%sql
SELECT * FROM cars_cleaned LIMIT 10;
```

settings

maker	model	mileage	manufacture_year	engine_displacement	engine_power	body_type	transr
ford	galaxy	151000	2011	2000	103	null	mar
skoda	octavia	143476	2012	2000	81	null	mar
bmw	null	97676	2010	1995	85	null	mar
skoda	fabia	111970	2004	1200	47	null	mar
skoda	fabia	128886	2004	1200	47	null	mar

Took 1 sec. Last updated by anonymous at February 12 2025, 2:33:29 PM. (outdated)

Creating clean_used_cars by Applying Filters

FINISHED

Took 0 sec. Last updated by anonymous at February 12 2025, 2:34:58 PM.

```
%sql
CREATE OR REPLACE TEMPORARY VIEW clean_used_cars AS
SELECT
  maker,
  model,
  mileage,
  manufacture_year,
  engine_displacement,
  engine_power,
  body_type,
  transmission,
  door_count,
  seat_count,
  price_eur,
  date_created,
  date_last_seen
FROM cars_cleaned
WHERE
  manufacture_year BETWEEN 2000 AND 2017
  AND maker IS NOT NULL AND maker != ''
  AND model IS NOT NULL AND model != ''
  AND price_eur BETWEEN 3000 AND 2000000;
```

FINISHED

Took 0 sec. Last updated by anonymous at February 12 2025, 2:36:51 PM.

```
%sql
SELECT COUNT(*) AS total_records
FROM clean_used_cars;
```

SPARK JOB FINISHED

Assignment-1-SparkSQL

1322853

--

Took 9 sec. Last updated by anonymous at February 12 2025, 2:38:37 PM. (outdated)

```
%sql
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
GROUP BY maker, model
ORDER BY avg_price DESC
LIMIT 10;
```

SPARK JOB FINISHED

maker	model	avg_price
lamborghini	aventador	365961.0
porsche	carrera-gt	302045.22
bmw	z8	245118.6
tesla	roadster	192880.28
tesla	model-x	176418.32
bentley	brooklands	138501.3
rolls-royce	wraith	137663.47
bentley	continental-gtc	129138.88

Took 41 sec. Last updated by anonymous at February 12 2025, 2:41:24 PM. (outdated)

Finding the Top 10 Make & Models with the Highest Average Prices

FINISHED

Took 0 sec. Last updated by anonymous at February 12 2025, 2:42:37 PM.

Assignment-1-SparkSQL

```
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price
LIMIT 10;
```



  settings ▼

maker	model	avg_price	
lamborghini	aventador	365961.0	▲
porsche	carrera-gt	302045.22	
bmw	z8	245118.6	
tesla	roadster	192880.28	
tesla	model-x	176418.32	
bentley	brooklands	138501.3	
rolls-royce	wraith	137663.47	
bentley	continental-gtc	129138.88	▼

Took 31 sec. Last updated by anonymous at February 12 2025, 2:44:34 PM. (outdated)

```
%md
### Finding the Top 10 Make & Models with the Lowest Average Prices
```

FINISHED

Finding the Top 10 Make & Models with the Lowest Average Prices

Took 0 sec. Last updated by anonymous at February 12 2025, 2:45:10 PM.

```
%sql
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
GROUP BY maker, model
ORDER BY avg_price ASC -- Sorting by lowest average price
LIMIT 10;
```

SPARK JOB FINISHED



  settings ▼

maker	model	avg_price	
skoda	galaxy	3071.8	▲
rover	streetwise	3187.85	
kia	retona	3200.25	
bmw	transit	3290.16	
chevrolet	alero	3305.37	
hyundai	santamo	3391.01	

opel	kadett	3405.48
fiat	128	3460.4

Assignment-1-SparkSQL

Took 28 sec. Last updated by anonymous at February 12 2025, 2:47:06 PM. (outdated)

%md
Recommending the Top 5 Make & Models for Economic Segment Customers

FINISHED

Recommending the Top 5 Make & Models for Economic Segment Customers

Took 0 sec. Last updated by anonymous at February 12 2025, 2:48:16 PM.

%sql
SELECT
 maker,
 model,
 ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
WHERE price_eur BETWEEN 3000 AND 20000 -- Economic segment price range
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;

SPARK JOB FINISHED

settings

maker	model	avg_price
volvo	241	19980.0
volvo	960	19306.14
toyota	gt86	18807.61
toyota	venza	18510.29
infinity	m30	18424.13

Took 20 sec. Last updated by anonymous at February 12 2025, 2:50:19 PM. (outdated)

%md
Recommending the Top 5 Make & Models for Intermediate Segment Customers

FINISHED

Recommending the Top 5 Make & Models for Intermediate Segment Customers

Took 0 sec. Last updated by anonymous at February 12 2025, 2:51:19 PM.

%sql
SELECT
 maker,
 model,
 ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
WHERE price_eur BETWEEN 20000 AND 300000 -- Intermediate segment price range

SPARK JOB FINISHED

```
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;
```

Assignment-1-SparkSQL

maker	model	avg_price
lamborghini	aventador	272901.63
bmw	z8	235838.12
tesla	model-x	176418.32
hyundai	matrix	143241.43
bentley	brooklands	138501.3

Took 28 sec. Last updated by anonymous at February 12 2025, 2:52:39 PM. (outdated)

```
%md
### Recommend the Top 5 Make & Models for Luxury Segment Customers
```

FINISHED

Recommend the Top 5 Make & Models for Luxury Segment Customers

Took 0 sec. Last updated by anonymous at February 12 2025, 2:53:53 PM.

```
%sql
SELECT
  maker,
  model,
  ROUND(AVG(price_eur), 2) AS avg_price
FROM clean_used_cars
WHERE price_eur BETWEEN 300000 AND 2000000 -- Luxury segment price range
GROUP BY maker, model
ORDER BY avg_price DESC -- Sorting by highest average price within this range
LIMIT 5;
```

SPARK JOB FINISHED

settings

maker	model	avg_price
mazda	323	1350749.44
volkswagen	golf	1179236.37
fiat	500	1125000.0
toyota	yaris	1111152.11
fiat	panda	1100000.0

Assignment-1-SparkSQL

FINISHED

```
%md
# Comparing Differences Between Hive and SparkSQL

## 1. Performance
- **SparkSQL** is faster for large datasets due to in-memory processing.
- **Hive** may take longer since it uses HDFS storage and MapReduce.

## 2. NULL Handling
- If Hive and SparkSQL treat "None" values differently, there could be small differences in NULL handling.

## 3. Data Type Inference
- SparkSQL automatically infers schema, whereas Hive requires explicit data types.
```

Comparing Differences Between Hive and SparkSQL

1. Performance

- SparkSQL is faster for large datasets due to **in-memory processing**.
- Hive may take longer since it uses **HDFS storage and MapReduce**.

2. NULL Handling

- If Hive and SparkSQL treat `"None"` values differently, there could be **small differences in NULL handling**.

3. Data Type Inference

- SparkSQL **automatically infers schema**, whereas Hive requires **explicit data types**.

%md

READY

1. Summary Report

1. Number of records in **used_cars** table:

   
total_records
3552912

2. Number of records in **clean_used_cars** table:

   
total_records
1322853

3. Make and model for the cars **with the top 10 highest average prices** and their average price:

settings

maker	model	avg_price
lamborghini	aventador	365961.0
porsche	carrera-gt	302045.22
bmw	z8	245118.6
tesla	roadster	192880.28
tesla	model-x	176418.32
bentley	brooklands	138501.3
rolls-royce	wraith	137663.47
bentley	continental-gtc	129138.88

4. Make and model for the cars **with the top 10 lowest average prices** and their average price:

maker	model	avg_price
skoda	galaxy	3071.8
rover	streetwise	3187.85
kia	retona	3200.25
bmw	transit	3290.16
chevrolet	alero	3305.37
hyundai	santamo	3391.01
opel	kadett	3405.48
fiat	128	3460.4
nissan	frontier	3478.9
seat	inca	3498.66

5. Write the name of the **top five makes and models** for **Economic** segment customers

maker	model	avg_price
volvo	241	19980.0
volvo	960	19306.14
toyota	gt86	18807.61
toyota	venza	18510.29
infinity	m30	18424.13

6. Write the name of the **top five makes and models** for **Intermediate** segment customers:

     	 ▼	settings ▼
maker	model	avg_price
lamborghini	aventador	272901.63
bmw	z8	235838.12
tesla	model-x	176418.32
hyundai	matrix	143241.43
bentley	brooklands	138501.3

7. Write the name of the **top five makes and models** for **Luxury** segment customers:

     	 ▼	settings ▼
maker	model	avg_price
mazda	323	1350749.44
volkswagen	golf	1179236.37
fiat	500	1125000.0
toyota	yaris	1111152.11
fiat	panda	1100000.0